

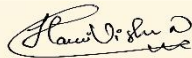
SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO1 ASSESSMENT TOOL 2 – Agile Sprint Planning & User Story Workshop

Course Code:	CSA10 / CSA1023	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 2 – Agile Sprint Planning & User Story Workshop
Weightage:	20%	Total Marks:	25
CO1 Statement:	Apply advanced methodologies and agile project management techniques to manage software projects effectively		
Student Name:	HARI VISHNU N	Reg. No.:	192524319
Date:	09-10-2026	Duration:	60 minutes

Code of Conduct

I HARI VISHNU N / 192524319 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:



Annexure A – Agile Sprint Planning & User Story Workshop

Instructions to Students:

Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

- 1. Identify the Project Vision and Stakeholders**
 - Define the project's vision, objectives, and identify all key stakeholders involved.
- 2. Create Epics and User Stories**
 - Break down the requirements into Epics and formulate well-structured User Stories using the format:
As a <user>, I want <feature>, so that <benefit>.
- 3. Define Acceptance Criteria**
 - Write clear and measurable acceptance criteria for each selected user story using the Given–When–Then format or equivalent.
- 4. Prioritize the Product Backlog**
 - Organize and prioritize the product backlog based on business value, customer needs, and project dependencies.
- 5. Plan the Sprint Backlog**
 - Select high-priority user stories for the first sprint and divide them in _____ to actionable development tasks.
- 6. Estimate Effort Using Story Points**
 - Assign story points to each selected user story using an appropriate estimation technique (e.g., Fibonacci sequence or Planning Poker) and justify your estimates.
- 7. Present the Sprint Goal and Expected Deliverables**
 - Define a clear sprint goal and describe the expected sprint deliverables, including the features to be completed at the end of the sprint.
 -

QUESTION

11. AI-Powered Requirements Conflict and Ambiguity Detection System

Industry Context:

Large software projects receive requirements from users, customers, business teams, developers, security teams, and regulators. Requirements may be incomplete, ambiguous, duplicated, or contradictory. These problems can remain hidden until design or implementation, resulting in costly rework.

Problem Statement:

Design and develop an AI-Powered Requirements Conflict and Ambiguity Detection System that analyses software requirements and identifies inconsistencies, ambiguity, duplication, missing dependencies, and conflicting stakeholder expectations.

The system should support collaborative documentation, requirements prioritization, traceability, and human validation. It should integrate with project-management tools and maintain version history while ensuring that AI recommendations remain reviewable and explainable.

Objectives

The system should:

- Analyse requirements for ambiguity and contradictions
- Detect duplicate and conflicting requirements
- Identify missing dependencies and incomplete information
- Support MoSCoW or similar prioritization techniques
- Maintain traceability between requirements and development tasks
- Provide collaborative review and version control
- Support explainable and human-validated AI recommendations

Expected Outcomes

The proposed system will:

- Reduce requirements-related rework
- Improve clarity and consistency of software specifications
- Support better stakeholder collaboration
- Improve requirements traceability
- Reduce project risks caused by conflicting requirements
- Strengthen requirements engineering practices

SOLUTION:

1. Identify the Project Vision and Stakeholders:

1.1 Project Vision:

The vision of the AI-Powered Requirements Conflict and Ambiguity Detection System is to develop an intelligent requirements engineering platform that uses Artificial Intelligence and Natural Language Processing (NLP) to analyse software requirements and automatically identify ambiguities, contradictions, duplicate requirements, missing dependencies, incomplete information, and stakeholder conflicts.

The system will help software teams improve the quality and consistency of requirements before development begins. It will also support requirements prioritization, collaborative review, traceability, version control, and explainable AI recommendations.

Project Objectives

The system aims to:

- Detect ambiguous and unclear requirements.
- Identify contradictory or conflicting requirements.
- Detect duplicate or highly similar requirements.
- Identify missing dependencies and incomplete information.
- Support MoSCoW prioritization.
- Maintain traceability between requirements and development tasks.
- Provide collaborative requirements review.
- Maintain requirements version history.
- Provide explainable AI-based recommendations.
- Keep human stakeholders involved in validating AI findings.

1.2 Stakeholders

Stakeholder	Role / Interest
Product Owner	Defines business requirements and prioritizes the backlog.
Business Analyst	Collects, analyses, and documents requirements.
Software Developer	Uses validated requirements for implementation.
Project Manager / Scrum Master	Manages project progress, sprint planning, and team coordination.

QA / Testing Team	Uses requirements to prepare test cases and verify system behaviour.
Security Team	Reviews security-related requirements and conflicts.
Customer / Client	Provides business needs and validates requirements.
End User	Provides user expectations and validates usability-related requirements.
Regulatory / Compliance Team	Ensures requirements satisfy applicable regulations and standards.
AI/ML Engineer	Develops and maintains AI/NLP-based detection models.
System Administrator	Manages users, access, configuration, and system availability.

2. Create Epics and User Stories

The requirements are divided into major Epics. Each Epic contains smaller User Stories.

2.1 Epic 1: Requirement Management

Purpose: Allow users to create, upload, edit, and manage software requirements.

User Story 1.1

As a Business Analyst, I want to add a new software requirement, so that I can maintain requirements in the system.

User Story 1.2

As a Business Analyst, I want to upload multiple requirements, so that I can analyse an existing requirements document efficiently.

User Story 1.3

As a Product Owner, I want to edit requirements, so that incorrect or outdated information can be corrected.

2.2 Epic 2: Ambiguity Detection

Purpose: Identify unclear, vague, or incomplete requirements using AI/NLP.

User Story 2.1

As a Business Analyst, I want the system to detect ambiguous words or phrases, so that unclear requirements can be corrected before development.

User Story 2.2

As a Developer, I want to see an explanation for an ambiguity warning, so that I can understand why the requirement is considered unclear.

2.3 Epic 3: Conflict and Contradiction Detection

Purpose: Identify requirements that contradict each other or represent conflicting stakeholder expectations.

User Story 3.1

As a Business Analyst, I want the system to compare requirements and identify contradictions, so that conflicts can be resolved before implementation.

User Story 3.2

As a Product Owner, I want to view conflicting requirements together, so that I can decide which requirement should be retained.

2.4 Epic 4: Duplicate and Dependency Detection

Purpose: Detect repeated requirements and missing relationships between requirements.

User Story 4.1

As a Business Analyst, I want the system to detect duplicate requirements, so that redundant requirements can be removed.

User Story 4.2

As a Developer, I want the system to identify missing requirement dependencies, so that implementation problems can be avoided.

2.5 Epic 5: Requirement Prioritization

Purpose: Prioritize requirements according to business importance.

User Story 5.1

As a Product Owner, I want to classify requirements using MoSCoW prioritization, so that high-value requirements are implemented first.

2.6 Epic 6: Traceability and Collaboration

Purpose: Connect requirements with development tasks and allow stakeholders to collaborate.

User Story 6.1

As a Project Manager, I want to trace requirements to development tasks, so that I can monitor implementation progress.

User Story 6.2

As a Stakeholder, I want to comment on and review requirements, so that requirements can be collaboratively validated.

2.7 Epic 7: Version Control and Explainable AI

Purpose: Maintain requirement history and make AI recommendations understandable.

User Story 7.1

As a Business Analyst, I want to view previous versions of a requirement, so that I can track changes over time.

User Story 7.2

As a Stakeholder, I want AI recommendations to include explanations, so that I can review and validate them before taking action.

3. Define Acceptance Criteria

The following acceptance criteria use the Given–When–Then format.

User Story 2.1 – Ambiguity Detection

User Story:

As a Business Analyst, I want the system to detect ambiguous words or phrases, so that unclear requirements can be corrected before development.

Acceptance Criteria

Given a requirement containing vague terms such as *fast*, *easy*, *efficient*, or *user-friendly*

When the requirement is analysed

Then the system should identify and highlight the ambiguous terms.

Given an ambiguous requirement

When the AI analysis is completed

Then the system should provide an explanation for why the requirement is ambiguous.

User Story 3.1 – Conflict Detection

User Story:

As a Business Analyst, I want the system to compare requirements and identify contradictions, so that conflicts can be resolved before implementation.

Acceptance Criteria

Given two requirements with contradictory conditions

When the requirements are analysed

Then the system should identify them as conflicting.

Given a detected conflict

When the user selects the conflict

Then the system should display the two conflicting requirements and explain the reason for the conflict.

User Story 4.1 – Duplicate Detection

User Story:

As a Business Analyst, I want the system to detect duplicate requirements, so that redundant requirements can be removed.

Acceptance Criteria

Given two or more requirements with substantially similar meanings

When the system analyses the requirements

Then it should identify them as potential duplicates.

Given a potential duplicate

When the user reviews the result

Then the system should display the similar requirements for comparison.

User Story 4.2 – Dependency Detection

User Story:

As a Developer, I want the system to identify missing requirement dependencies, so that implementation problems can be avoided.

Acceptance Criteria

Given a requirement that depends on another requirement

When the requirements are analysed

Then the system should identify the dependency.

Given a missing dependency

When the analysis is completed

Then the system should generate a warning for the stakeholder.

User Story 5.1 – MoSCoW Prioritization

User Story:

As a Product Owner, I want to classify requirements using MoSCoW prioritization, so that high-value requirements are implemented first.

Acceptance Criteria

Given a list of requirements

When the Product Owner prioritizes them

Then each requirement should be assigned one of Must Have, Should Have, Could Have, or Won't Have.

Given prioritized requirements

When the backlog is displayed

Then the requirements should be ordered according to their priority.

User Story 6.1 – Requirement Traceability

User Story:

As a Project Manager, I want to trace requirements to development tasks, so that I can monitor implementation progress.

Acceptance Criteria

Given a validated requirement

When a development task is created

Then the task should be linked to the corresponding requirement.

Given a requirement linked to a development task

When the user views the requirement

Then the associated task and its status should be displayed.

User Story 7.2 – Explainable AI

User Story:

As a Stakeholder, I want AI recommendations to include explanations, so that I can review and validate them before taking action.

Acceptance Criteria

Given an AI-generated conflict or ambiguity warning

When the stakeholder opens the recommendation

Then the system should display the reason for the recommendation.

Given an AI recommendation

When the stakeholder reviews it

Then the stakeholder should be able to accept, reject, or modify the recommendation.

4. Prioritize the Product Backlog

The backlog is prioritized using business value, customer need, risk, and dependencies.

Priority Levels

- **P1 – High:** Essential for the core system.
- **P2 – Medium:** Important but can follow core functionality.
- **P3 – Low:** Useful additional functionality.

ID	Epic	User Story	Priority	Reason
US-01	Requirement Management	Add software requirement	P1	Foundation of the system
US-02	Requirement Management	Upload multiple requirements	P1	Required for practical analysis
US-03	Ambiguity Detection	Detect ambiguous requirements	P1	Core AI functionality
US-04	Conflict Detection	Detect contradictory requirements	P1	Core project objective
US-05	Duplicate Detection	Detect duplicate requirements	P1	Reduces redundancy
US-06	Dependency Detection	Detect missing dependencies	P1	Prevents implementation issues
US-07	Explainable AI	Explain AI recommendations	P1	Required for human validation
US-08	Prioritization	MoSCoW prioritization	P2	Supports backlog management
US-09	Traceability	Link requirements to tasks	P2	Improves project tracking
US-10	Collaboration	Review and comment	P2	Supports stakeholder collaboration
US-11	Version Control	Maintain requirement history	P2	Supports change management
US-12	Integration	Integrate with project-management tools	P3	Useful for future enhancement

5. Plan the Sprint Backlog

For Sprint 1, the focus should be on establishing the foundation and implementing the most important AI analysis capabilities.

Sprint 1 – Selected User Stories

ID	User Story	Development Tasks
US-01	Add software requirement	Design requirement form; create requirement database; implement add/edit functionality
US-02	Upload multiple requirements	Implement document upload; extract requirement text; validate input
US-03	Detect ambiguous requirements	Develop NLP preprocessing; identify vague terms; generate ambiguity warnings
US-04	Detect contradictory requirements	Implement requirement comparison; develop conflict detection logic; display conflicts
US-05	Detect duplicate requirements	Implement text similarity; compare requirement pairs; display duplicate warnings
US-07	Explain AI recommendations	Generate explanation for detected issues; create review interface

Sprint 1 Development Tasks

Task 1: Design the system architecture and database.

Task 2: Create the requirement entry and upload interface.

Task 3: Implement requirement text preprocessing.

Task 4: Implement AI-based ambiguity detection.

Task 5: Implement requirement similarity analysis.

Task 6: Implement conflict detection.

Task 7: Develop duplicate requirement detection.

Task 8: Develop the AI explanation/recommendation interface.

Task 9: Perform unit and integration testing.

Task 10: Conduct stakeholder review and validation.

6. Estimate Effort Using Story Points:

The Fibonacci sequence is used for estimation:

1, 2, 3, 5, 8, 13

Higher points indicate greater complexity, uncertainty, and development effort.

ID	User Story	Story Points	Justification
US-01	Add software requirement	3	Basic CRUD functionality
US-02	Upload multiple requirements	5	Requires file processing and text extraction
US-03	Detect ambiguous requirements	8	Requires NLP and AI analysis
US-04	Detect contradictory requirements	13	Complex semantic comparison and conflict analysis
US-05	Detect duplicate requirements	8	Requires text similarity/semantic comparison
US-06	Detect missing dependencies	8	Requires relationship and dependency analysis
US-07	Explain AI recommendations	5	Requires explainability and review interface
US-08	MoSCoW prioritization	3	Relatively straightforward classification
US-09	Requirement-task traceability	5	Requires linking and tracking
US-10	Collaborative review	5	Requires comments and stakeholder interaction
US-11	Version control	5	Requires history and change tracking
US-12	Project-management integration	8	Requires external API integration

Sprint 1 Story Point Calculation

Selected Sprint 1 stories:

- US-01 = 3
- US-02 = 5

- US-03 = 8
- US-04 = 13
- US-05 = 8
- US-07 = 5

Total = 42 Story Points

For an academic Sprint 1 plan, this represents the planned workload. In an actual Scrum project, the final commitment would depend on the team's historical velocity and available capacity.

7. Present the Sprint Goal and Expected Deliverables:

Sprint 1 Goal

"Develop the foundational requirements management and AI analysis capabilities that allow stakeholders to enter or upload software requirements and automatically identify ambiguity, conflicts, and duplicate requirements with explainable recommendations."

Expected Sprint 1 Deliverables

At the end of Sprint 1, the system is expected to provide:

1. Requirement Management Module
 - Add requirements.
 - Edit requirements.
 - Store requirements.
2. Requirement Upload Module
 - Upload requirements documents.
 - Extract requirement text.
3. AI Ambiguity Detection
 - Identify vague or ambiguous terms.
 - Highlight problematic requirements.
 - Provide explanations.
4. Conflict Detection
 - Compare requirements.
 - Detect contradictory requirements.
 - Display conflicting requirements.
5. Duplicate Detection
 - Identify similar requirements.
 - Display possible duplicates.

6. Explainable AI

- Show why a requirement was flagged.
- Allow stakeholders to review AI recommendations.

7. Testing and Validation

- Test AI detection functions.
- Validate results with stakeholders.
- Record identified issues for the next sprint.

Overall Product Backlog:

For the complete project, the Agile backlog can be represented as:

Rank	User Story	Priority	Story Points	Sprint
1	Add software requirements	Must Have	3	Sprint 1
2	Upload requirements documents	Must Have	5	Sprint 1
3	Detect ambiguous requirements	Must Have	8	Sprint 1
4	Detect contradictory requirements	Must Have	13	Sprint 1
5	Detect duplicate requirements	Must Have	8	Sprint 1
6	Explain AI recommendations	Must Have	5	Sprint 1
7	Detect missing dependencies	Must Have	8	Sprint 2
8	MoSCoW prioritization	Should Have	3	Sprint 2
9	Requirement-task traceability	Should Have	5	Sprint 2
10	Collaborative review and comments	Should Have	5	Sprint 2
11	Requirement version history	Should Have	5	Sprint 3
12	Project-management tool integration	Could Have	8	Sprint 3

Sprint Roadmap

Sprint	Main Focus	Expected Outcome
Sprint 1	Requirement management + AI ambiguity/conflict/duplicate detection	Working AI analysis prototype
Sprint 2	Dependency detection + prioritization + traceability + collaboration	Complete requirements analysis and management workflow
Sprint 3	Version control + project-management integration + refinement	Integrated and production-ready system

Conclusion:

The proposed AI-Powered Requirements Conflict and Ambiguity Detection System applies Agile principles by converting the problem statement into Epics, User Stories, Acceptance Criteria, a prioritized Product Backlog, and a Sprint Backlog. Sprint 1 focuses on the core functionality of collecting requirements and using AI to detect ambiguity, conflicts, and duplication, while also providing explainable recommendations.

The planned system will ultimately help organizations reduce requirements-related rework, improve specification quality, strengthen stakeholder collaboration, and maintain traceability throughout the software development lifecycle.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Project Vision & Stakeholder Identification (30%)	Project vision is clear, comprehensive, and aligned with the problem statement. All relevant stakeholders and their roles are correctly identified.	Vision is clear with minor omissions. Most stakeholders are correctly identified.	Vision is partially defined. Some stakeholders are missing or incorrectly identified.	Vision is unclear or incomplete. Stakeholders are poorly identified or missing.
Epics & User Stories (25%)	Epics are well-defined and user stories are complete, follow Agile format, and fully capture functional requirements.	Epics and user stories are mostly complete with minor formatting or requirement issues.	User stories are partially complete with limited Agile structure.	User stories are incomplete, unclear, or do not follow Agile practices.
Acceptance Criteria (20%)	Acceptance criteria are specific, measurable, testable, and aligned with all user stories.	Acceptance criteria are mostly clear and testable with minor gaps.	Acceptance criteria are partially defined and lack clarity or completeness.	Acceptance criteria are missing or not aligned with user stories.
Product Backlog Prioritization & Sprint Planning (15%)	Product backlog is logically prioritized, sprint backlog is realistic, and task breakdown is complete.	Backlog prioritization and sprint planning are mostly appropriate with minor issues.	Prioritization or sprint planning is partially complete with limited justification.	Backlog prioritization and sprint planning are poorly organized or incomplete.
Story Point Estimation, Sprint Goal & Presentation (10%)	Story point estimation is well justified, sprint goal is clear, deliverables are realistic, and presentation is professional and well organized.	Estimation and sprint goal are mostly appropriate with minor inconsistencies. Presentation is clear.	Estimation or sprint goal lacks justification. Presentation is adequate but incomplete.	Estimation is unrealistic or missing. Sprint goal, deliverables, and presentation are unclear or incomplete.

Criteria	Mark
Project Vision & Stakeholder Identification (30%)	/5
Epics & User Stories (25%)	/5
Acceptance Criteria (20%)	/5
Product Backlog Prioritization & Sprint Planning (15%)	/5
Story Point Estimation, Sprint Goal & Presentation (10%)	/5
TOTAL	/25